

CUADERNO DE BITACORA

Proyecto de entrenamiento y evaluacion de un modelo de lenguaje

Proyecto 50M_LLM	Periodo _____
Responsable _____	Version del cuaderno v0.1

Registra decisiones, evidencias y aprendizajes mientras trabajas. Al final, cada entrada se convierte en material directo para el informe tecnico.

Como usar este cuaderno

Una entrada breve y honesta por cada bloque de trabajo importante vale mas que reconstruir la historia al final. Escribe primero lo que observaste; despues tu interpretacion.

Cuando	Que anotar	Para el informe final
Antes de ejecutar	Hipotesis, configuracion y criterio de exito.	Metodologia y disenio experimental.
Durante	Cambios, incidencias, coste y observaciones.	Implementacion y limitaciones.
Despues	Metricas, comparacion con el baseline y conclusion.	Resultados y discusion.
Al decidir	Alternativas descartadas y por que.	Justificacion tecnica.

Regla practica

1. Fecha y contexto	2. Evidencia verificable	3. Decision y siguiente paso
---------------------	--------------------------	------------------------------

Convencion sugerida

Nombra los experimentos con fecha y objetivo: **2026-09-11_lr-sweep_v1**. Guarda junto a cada entrada enlaces o rutas a graficas, checkpoints, notebooks y ejecuciones.

Ficha del proyecto

Problema que resuelve	Objetivo medible
Hipotesis inicial	Alcance y exclusiones
Datos: fuente, licencia, tamaño, limpieza	Métricas principales y baseline
Recursos: hardware, tiempo, presupuesto	Riesgos técnicos / éticos

Configuracion comun hasta V4.3.3

Estos valores se han mantenido constantes en las iteraciones documentadas hasta ahora. Constituyen el contexto comun para interpretar las mejoras incrementales.

Parametro	Valor fijo	Implicacion
SEED	123	Reduce variacion de una corrida a otra; una sola seed no mide robustez estadistica.
MAX_LENGTH	128 tokens	Longitud maxima de contexto por secuencia.
BATCH_SIZE	2	Tamano de lote por actualizacion antes de cualquier acumulacion no registrada.
MAX_TOKENS	1,000,000	Presupuesto total de tokens de entrenamiento.
MAX_UPDATES	3,906	1,000,000 // (2 x 128): numero maximo de actualizaciones previsto.
LEARNING_RATE	3e-4	Pico/base de learning rate; desde V4.2 se planifica con warm-up y cosine decay.
WEIGHT_DECAY	0.1	Regularizacion aplicada con AdamW.
EVAL_INTERVAL	10 updates	Frecuencia de evaluacion de validacion.
VAL_BATCHES	10	Numero de batches usados por cada evaluacion de validacion.

Nota de comparabilidad

Mantener este bloque fijo hace las comparaciones posteriores mas interpretables. Aun asi, las versiones V4.1 a V4.3.3 incorporan cambios de forma acumulativa; por tanto, este control no convierte por si solo toda la trayectoria en un estudio de ablacion.

Entrada de bitacora 01

Implementacion V4.1 - baseline de GPT

Fecha: no registrada Autor/a: _____ Estado: primera implementacion

Objetivo

Validar un pipeline inicial de preentrenamiento autoregresivo: arquitectura Transformer tipo GPT, tokenizador GPT-2 ya disponible y entrenamiento de una epoca. Esta ejecucion funciona como baseline para comparar iteraciones posteriores.

Configuracion registrada

Arquitectura Transformer basico tipo GPT.	Tokenizacion Tokenizador GPT-2 ya cargado.
Tamano maximo del modelo Aprox. 47,85 M parametros.	Optimizador AdamW.
Entrenamiento 1 epoca sobre aprox. 1 M de tokens.	Evidencia disponible Curva de loss de entrenamiento y validacion.

Resultado observado

El loss de validacion comunicado para esta ejecucion es **7,47**. La grafica adjunta anota una referencia final de **7,40711684**; ambas cifras quedan registradas tal como se han recibido y conviene confirmar en el log original cual corresponde al checkpoint final.

Interpretacion y aprendizajes

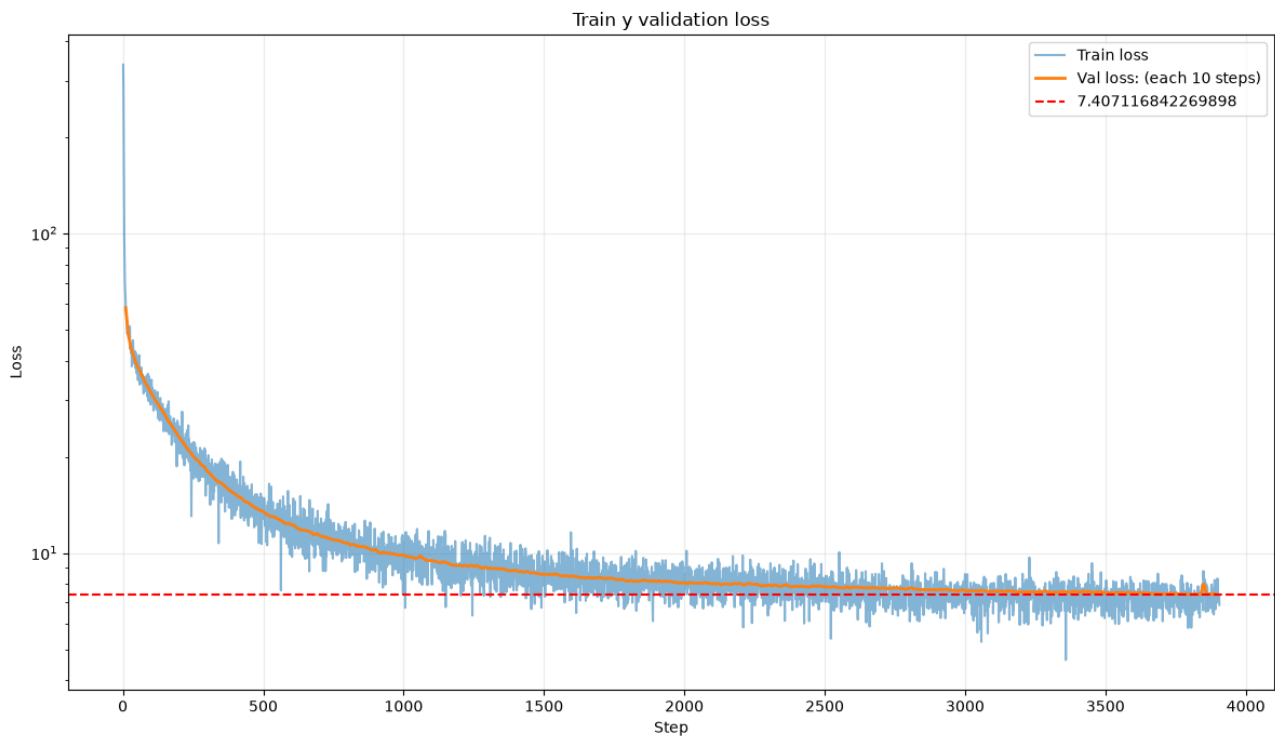
La curva de entrenamiento y la de validacion descienden de forma sostenida y permanecen proximas al final, por lo que esta ejecucion confirma que el pipeline aprende una senal util. El volumen de entrenamiento es muy reducido frente al tamano del modelo, así que el resultado debe tratarse como un baseline funcional, no como una evaluacion de capacidad final. El pico inicial de train loss merece revisarse en futuras corridas si vuelve a aparecer.

Siguiente paso propuesto

Conservar configuracion, semilla, version de datos y checkpoint. Aumentar tokens de entrenamiento y comparar contra este baseline con las mismas metricas y un presupuesto de parametros controlado.

Evidencia - V4.1

Curva de loss de entrenamiento y validacion aportada para la primera implementacion.



Lectura de la figura

La validacion baja rapidamente al principio y despues se estabiliza de manera gradual. Al final, train y validacion se encuentran en el rango aproximado de 7 a 8 de loss.

Nota de trazabilidad

La linea discontinua del grafico esta etiquetada como 7.407116842269898. Para el informe final, confirmar si es el loss de validacion final, un promedio o el mejor valor guardado.

Entrada de bitacora 02

Implementacion V4.2 - cosine learning rate decay

Fecha: no registrada Autor/a: _____ Estado: comparacion con V4.1

Cambio introducido

Se incorpora un scheduler de learning rate con **warm-up** y **cosine decay**. El learning rate crece hasta aproximadamente $3e-4$ durante las primeras actualizaciones y despues disminuye suavemente hasta el final de la corrida.

Condiciones de comparacion

Modelo de referencia GPT basico de V4.1.	Presupuesto de datos Aprox. 1 M de tokens.
Parametro cambiado Planificacion del learning rate: cosine decay.	Lecturas comparadas Train loss en escala logaritmica y validation loss.
Resultado comunicado Validation loss: 7.60 -> 7.40.	Diferencia -0.20 puntos; mejora aproximada del 2.6% frente al baseline comunicado.

Interpretacion

El cosine decay produce una mejora pequena pero consistente con un presupuesto de solo 1 M de tokens: la diferencia en validacion es visible, aunque no permite extraer conclusiones fuertes sobre el regimen a largo plazo. La escala logaritmica del train loss hace mas legible la separacion entre las curvas durante la convergencia; la validacion debe seguir siendo la metrica principal para escoger el cambio.

Decision

Mantener el scheduler como candidato para las siguientes corridas y repetir la comparacion con mas tokens, mismas semillas y mismo split de validacion. Reportar el valor final, el mejor valor y la varianza entre seeds.

Nota de trazabilidad

Esta comparacion usa 7.60 como baseline y 7.40 como resultado con scheduler, segun el registro aportado. La entrada V4.1 contiene otros valores historicos (7.47 y 7.407 en una grafica); antes del informe final hay que verificar que todas las cifras se refieren al mismo split, checkpoint y metodo de agregacion.

Evidencia - V4.2

Planificación del learning rate y comparación de train loss con V4.1.

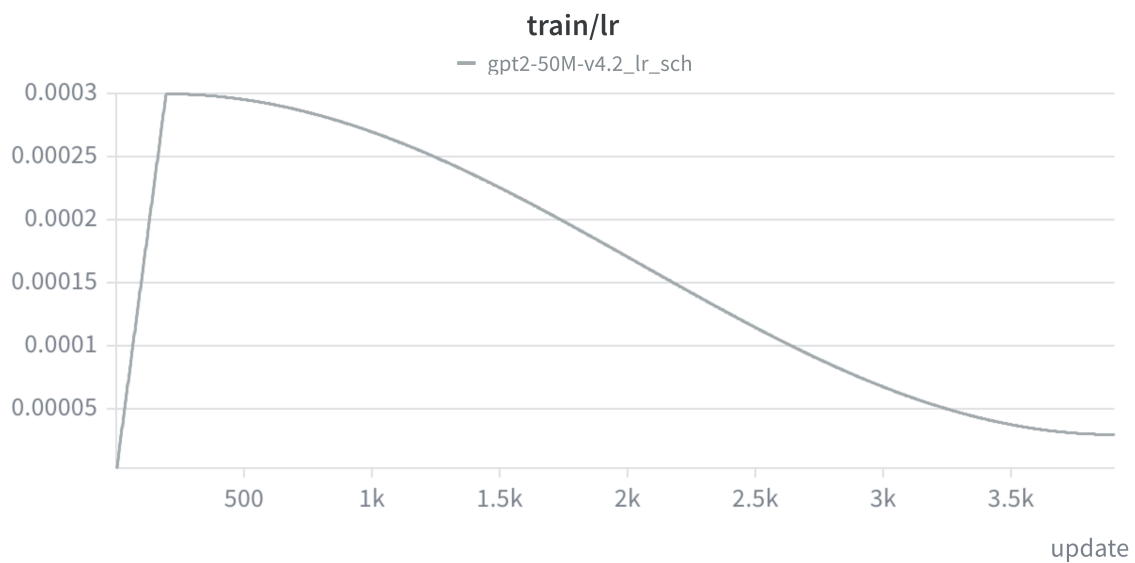


Figura 1. Warm-up inicial hasta aproximadamente $3e-4$, seguido de un descenso suave de tipo coseno. Los valores se estiman visualmente a partir de la grafica.

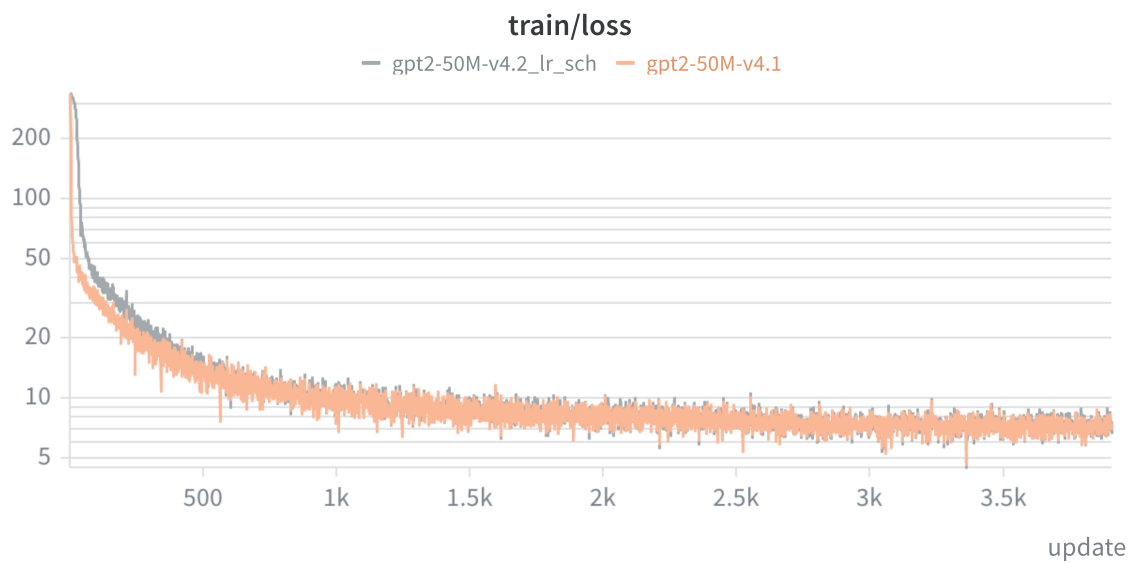
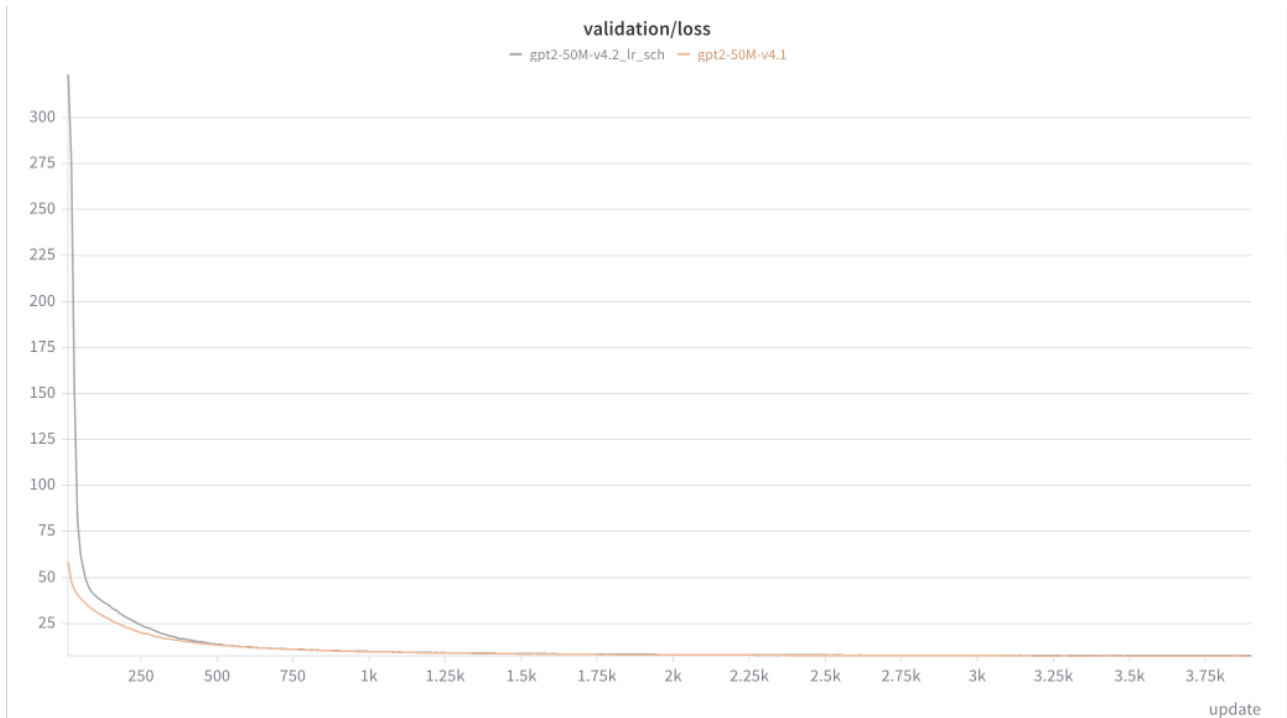


Figura 2. Train loss de V4.2 con scheduler frente a V4.1 en escala logaritmica. Esta escala permite comparar con mas claridad las diferencias durante la fase de convergencia.

Evidencia - V4.2 (validacion)

Comparacion de validation loss entre el scheduler y V4.1.



Resultado a conservar

El registro de la iteracion indica una reduccion de validation loss de 7.60 a 7.40 tras introducir cosine decay. La diferencia es limitada, algo esperable al entrenar solo 1 M de tokens, pero justifica realizar una corrida mas larga y controlada.

Pregunta para la siguiente iteracion

Con un mayor presupuesto de tokens, el scheduler mantiene o amplifica la ventaja frente al baseline? La respuesta exige comparar con las mismas semillas, datos y configuracion salvo el scheduler.

Entrada de bitacora 03

Implementacion V4.3.2 - inicializacion Xavier

Fecha: no registrada Autor/a: _____ Estado: mejora acumulativa sobre V4.2

Cambio introducido

Se sustituye la inicializacion previa por **inicializacion Xavier**, manteniendo las mejoras acumuladas hasta V4.2, incluido el scheduler de cosine decay.

Resultado observado

Loss inicial La curva muestra una caida muy marcada al inicio: de aproximadamente 250 en V4.2 a aproximadamente 10 con Xavier. Es una reduccion visible de mas de un orden de magnitud (aprox. 25x).	Validation loss El registro indica una reduccion aproximada de un punto en el conjunto de validacion/test respecto a la version anterior.
Train loss La curva con Xavier parte mucho mas baja y se mantiene por debajo de la referencia V4.2 durante la mayor parte de la corrida.	Lectura operativa La configuracion acumulada resulta mas estable y alcanza una mejor region de loss desde las primeras actualizaciones.

Limitacion metodologica importante

Diseno incremental, no ablacion aislada. Las modificaciones se han introducido acumulativamente para controlar el coste computacional y explorar rapido las opciones prometedoras. Por ello, la mejora observada en V4.3.2 no puede atribuirse de forma causal solo a Xavier: podria interactuar con el scheduler u otros cambios heredados. El resultado es valido como evidencia de una configuracion mejorada, no como una estimacion limpia del efecto individual de Xavier.

Decision y siguiente paso

Conservar Xavier en la configuracion operativa por su ganancia practica. Si hay presupuesto, ejecutar una ablacion con misma semilla, datos, scheduler y numero de tokens, cambiando unicamente la inicializacion. En el informe, presentar esta fase como optimizacion iterativa condicionada por recursos.

Evidencia - V4.3.2

Comparacion de Xavier frente a V4.2 con scheduler. Naranja: Xavier; gris: V4.2.

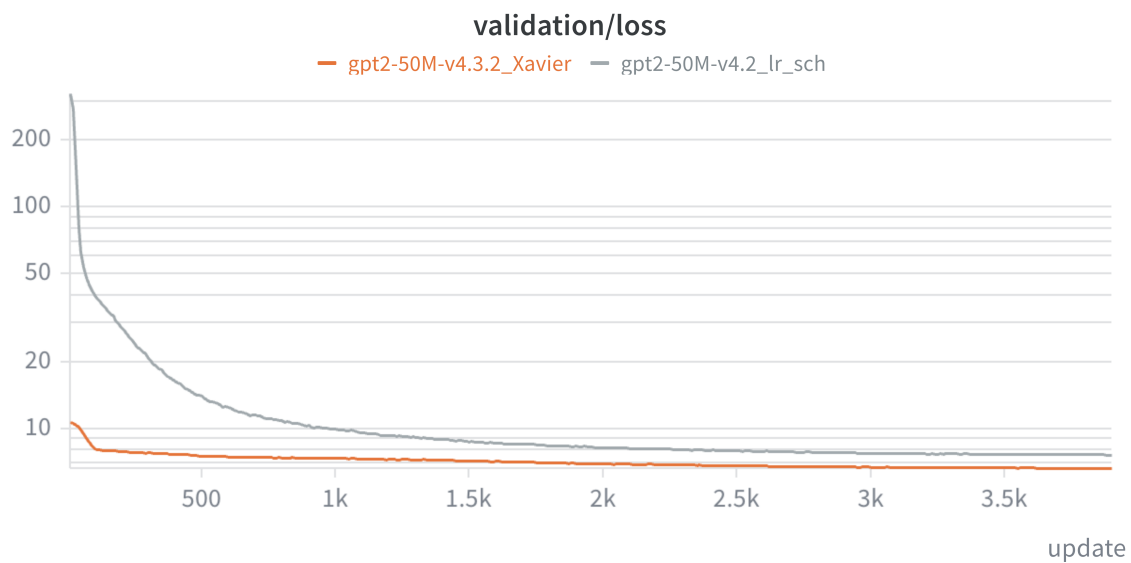


Figura 1. Validation loss. Xavier inicia en una escala mucho menor y termina aproximadamente un punto por debajo de V4.2, según el registro aportado.

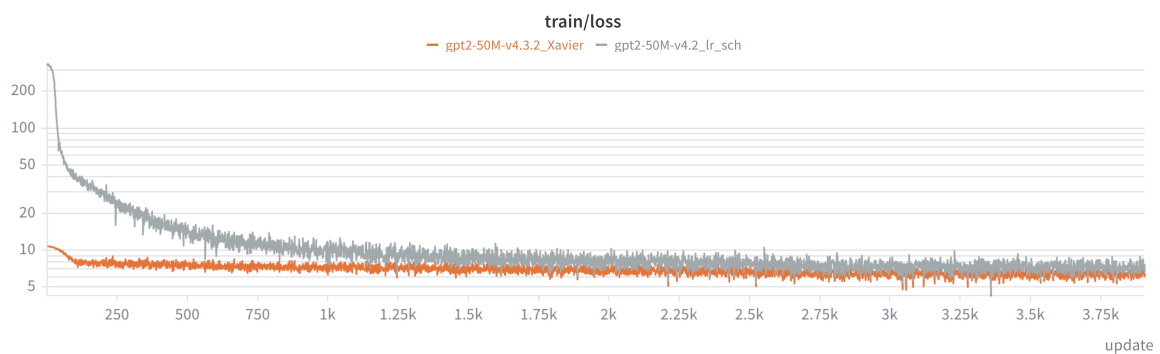


Figura 2. Train loss. La diferencia inicial es sustancial; esta comparacion debe interpretarse como el rendimiento de una configuracion acumulada, no como una ablacion de Xavier aislada.

Entrada de bitacora 04

Implementacion V4.3.3 - activacion SwiGLU

Fecha: no registrada Autor/a: _____ Estado: cambio sobre la configuracion V4.3.2

Cambio introducido

Se reemplaza la activacion anterior por **SwiGLU** en el bloque MLP del Transformer. El resto de la configuracion se mantiene segun el registro. En la terminologia del informe se usara SwiGLU, no 'Swish-GELU': es una unidad lineal con compuerta basada en Swish.

Motivacion tecnica

SwiGLU introduce una compuerta multiplicativa dentro del MLP, lo que aumenta la flexibilidad de la transformacion no lineal. La ventaja no procede de que la funcion de activacion tenga parametros entrenables por si sola, sino de la parametrizacion y expresividad del bloque con compuerta. Es una eleccion frecuente en arquitecturas modernas de modelos de lenguaje.

Resultado observado

Convergencia inicial Las curvas son muy proximas en las primeras actualizaciones; no aparece una ventaja dominante al inicio.	Mejora a largo plazo La curva de validation loss de SwiGLU empieza a separarse favorablemente tras mas actualizaciones y conserva una ventaja al final de la corrida.
Metrica relevante El test/validation loss general mejora respecto a Xavier, aunque la figura no aporta un valor final exacto para cuantificar la diferencia.	Interpretacion El cambio parece mas beneficioso cuando el modelo ha tenido suficiente numero de sub-batches para explotar la mayor expresividad del MLP.

Calidad de la comparacion

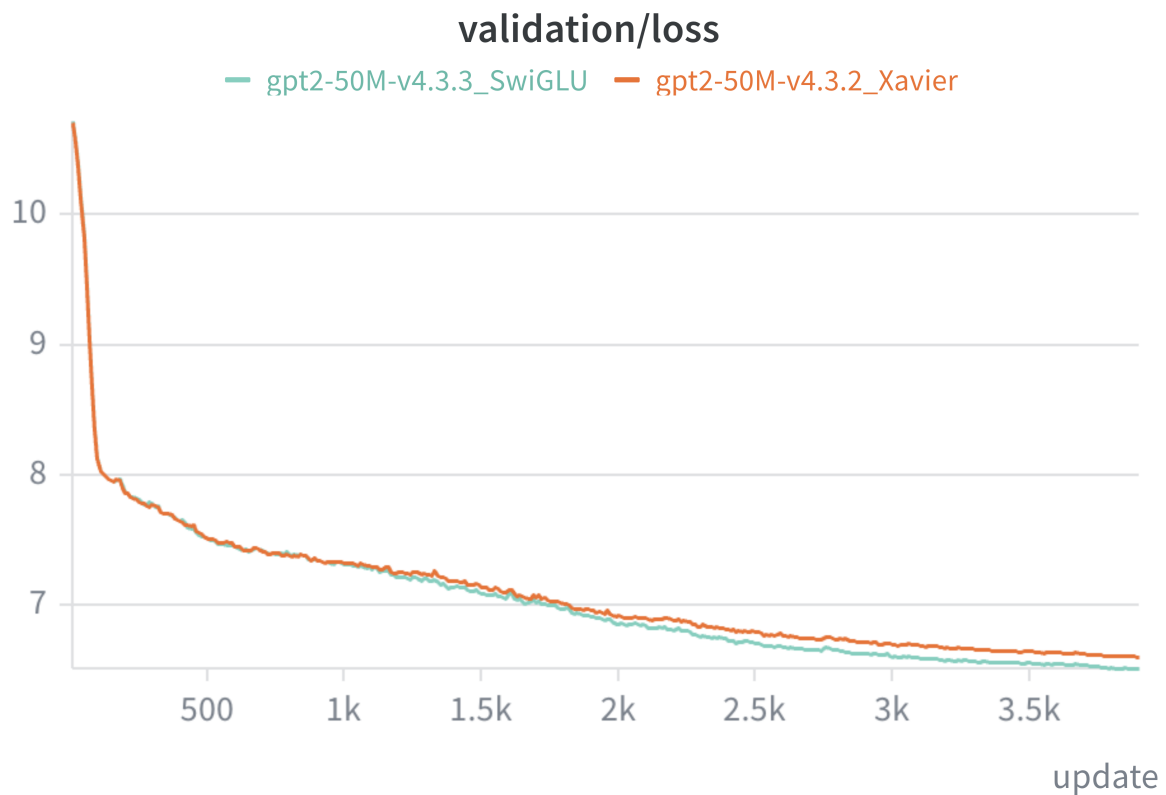
La trayectoria completa V4.1 -> V4.3.3 sigue siendo incremental y condicionada por recursos. Sin embargo, esta comparacion concreta es mas cercana a una ablacion que las anteriores si se confirma que datos, seed, scheduler, arquitectura y numero de tokens permanecieron iguales, cambiando solo la activacion. Esa verificacion debe constar en el informe final.

Decision y siguiente paso

Mantener SwiGLU como activacion candidata para corridas mas largas. Guardar los valores exactos de loss final y mejor checkpoint, y repetir la comparacion en varias seeds si el presupuesto lo permite.

Evidencia - V4.3.3

Validation loss: SwiGLU frente a la configuración con Xavier. Verde: SwiGLU; naranja: Xavier.



Lectura de la figura

Las dos curvas empiezan casi solapadas. La ventaja de SwiGLU se hace visible después de un mayor número de actualizaciones y crece gradualmente hasta el final, lo que apoya evaluar activaciones con presupuestos de entrenamiento suficientes.

Limitación

La gráfica no muestra una tabla de valores exactos ni intervalos entre seeds. La conclusión es direccional: SwiGLU mejora el validation loss observado en esta corrida; la magnitud y reproducibilidad siguen por confirmar.

Entrada de bitacora 05

Implementacion V4.4 - mayor profundidad

Fecha: no registrada Autor/a: _____ Estado: descartada

Hipotesis y cambio propuesto

Se prueba un Transformer mas profundo para aumentar la capacidad de modelado. La hipotesis era que mas capas mejorarian el validation/test loss manteniendo los demas componentes de la configuracion operativa.

Configuracion efectiva de V4.4

Campo	Valor efectivo	Nota
vocab_size	50,257	Sin cambio registrado.
context_length	128	El cfg final sobrescribe el valor 256 de GPT_CONFIG_50M con MAX_LENGTH.
emb_dim	480	Sin cambio registrado.
n_layers	9	Antes: 7. Este es el cambio de profundidad declarado.
n_heads	4	Antes: 8. Cambio simultaneo que impide aislar solo la profundidad.
ff_activation	SwiGLU	Se conserva.
ff_hidden_dim	1,376	El cfg final sobrescribe el 1,280 de la configuracion base.
drop_rate / qkv_bias	0.0 / False	Sin cambio registrado.

Resultado observado

La curva de validation loss de V4.4 (naranja) se mantiene por encima de la referencia V4.3.3 con SwiGLU (verde) durante la mayor parte del entrenamiento y termina peor. Bajo este presupuesto de tokens, la mayor profundidad no mejoro el test/validation loss y se descarta como configuracion operativa.

Interpretacion y limitacion

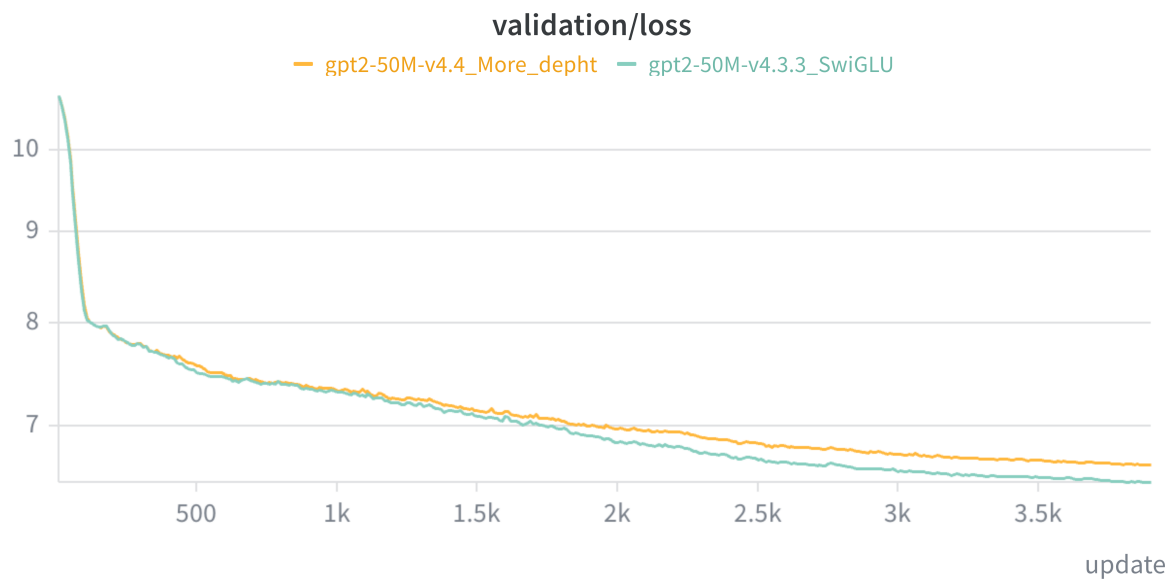
El resultado no demuestra que los modelos mas profundos sean intrinsecamente peores: con 1 M de tokens y el mismo regimen de entrenamiento, la capacidad adicional puede requerir mas optimizacion o datos. Ademias, n_layers y n_heads cambiaron a la vez (7 -> 9 y 8 -> 4), por lo que no es una ablacion limpia de profundidad.

Decision

Descartar V4.4 para la linea principal y conservar V4.3.3 como referencia. Si se retoma esta hipotesis, comparar 7 frente a 9 capas manteniendo fijo el numero de heads, la dimension de embedding, la anchura del MLP y el presupuesto de tokens.

Evidencia - V4.4

Validation loss: mayor profundidad frente a V4.3.3 con SwiGLU. Naranja: V4.4; verde: V4.3.3.



Lectura de la figura

Las curvas parten muy cerca, pero V4.4 empieza a quedarse por encima de la referencia al avanzar el entrenamiento y termina con peor validation loss. Esto sustenta la decisión de descartar la variante dentro del presupuesto evaluado.

Aprendizaje reutilizable

Al aumentar la profundidad, controlar las demás variables es especialmente importante. Para una ablacion interpretable, no modificar a la vez el número de heads ni otros componentes de capacidad.

Entrada de bitacora 06

Implementacion V4.5 - Looped Transformer

Fecha: no registrada Autor/a: _____ Estado: prometedora, pendiente de barrido

Idea y motivacion

Se implementa un Looped Transformer: un bloque o conjunto de bloques se reutiliza varias veces sobre la representacion. Esto aumenta el numero de transformaciones efectivas y el computo por ejemplo sin introducir un bloque independiente de parametros por cada repeticion. La idea se exploró por su interes arquitectonico; referencias externas no verificadas se consideran solo inspiracion, no evidencia sobre otros modelos.

Trade-off de capacidad

Parametros La reutilizacion de pesos puede aumentar la profundidad efectiva sin un crecimiento proporcional de parametros unicos.	Computo y latencia Cada recurrence adicional requiere mas operaciones y puede aumentar el tiempo de entrenamiento e inferencia.
Hipotesis Mas pasos recurrentes permiten refinar la representacion y mejorar el validation/test loss dentro del mismo presupuesto de parametros.	Variable pendiente Numero de recurrencias: aun no se ha determinado el valor optimo.

Resultado observado

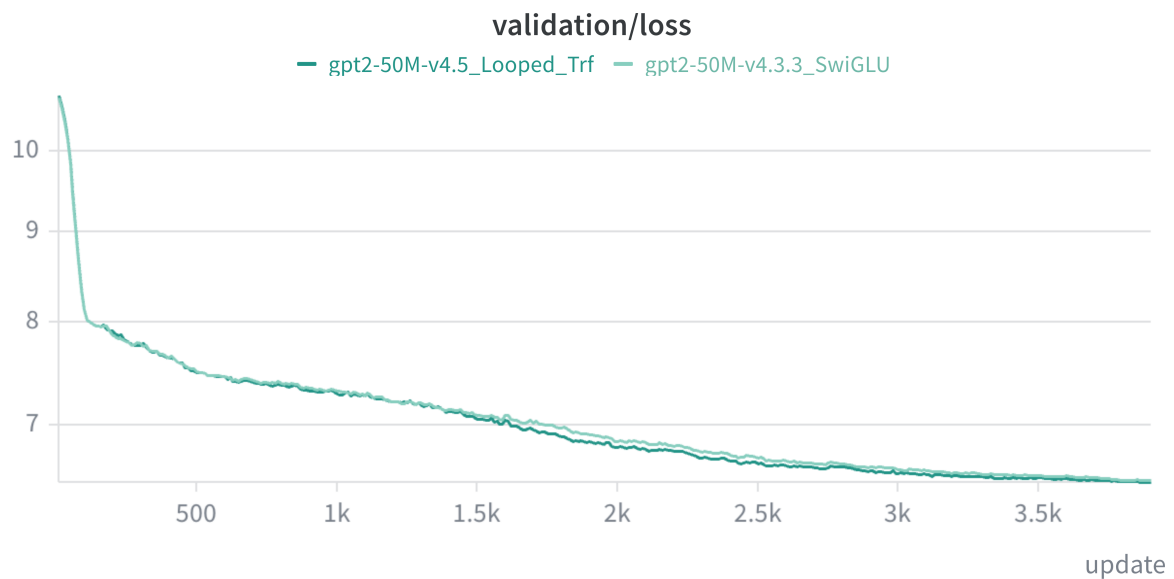
La curva de V4.5 (verde oscuro) muestra una mejora ligera de validation/test loss frente a V4.3.3 con SwiGLU (verde claro), especialmente durante la parte media del entrenamiento. Al final las curvas convergen, por lo que la ganancia observada es moderada y debe cuantificarse con valores exactos antes de afirmarla como mejora definitiva.

Decision y siguiente paso

Mantener Looped Transformer como linea de exploracion. Realizar un barrido del numero de recurrencias, registrando validation loss final, mejor checkpoint, tiempo por update, memoria y coste total. Elegir el valor optimo por calidad bajo un presupuesto de computo, no solo por numero de parametros.

Evidencia - V4.5

Validation loss: Looped Transformer frente a V4.3.3 con SwiGLU. Verde oscuro: Looped; verde claro: SwiGLU.



Lectura de la figura

La variante recurrente se sitúa ligeramente por debajo de la referencia durante una parte importante del entrenamiento. La diferencia se reduce hacia el final, por lo que la ventaja es prometedora pero todavía no concluyente sin valores exactos y repeticiones.

Experimento recomendado

Probar una cuadrícula pequeña de recurrencias, por ejemplo 1, 2, 3 y 4, manteniendo fijos datos, seed y presupuesto de tokens. Para cada punto, comparar calidad y coste: loss, tiempo por update, memoria y tokens por segundo.

Entrada de bitacora 07

Implementacion V4.7 - Rotary Positional Embeddings (RoPE)

Fecha: no registrada Autor/a: _____ Estado: mejora clara observada

Cambio introducido

Se reemplazan los positional embeddings por **Rotary Positional Embeddings (RoPE)**. En RoPE, la informacion de posicion se aplica mediante rotaciones a las representaciones de query y key en atencion, en lugar de sumarse como un embedding posicional aprendido independiente.

Motivacion tecnica

Posicion relativa La interaccion entre tokens puede codificar relaciones de posicion de forma natural en el mecanismo de atencion.	Parametros No requiere una tabla aprendida de posiciones del mismo modo que los positional embeddings absolutos.
Compatibilidad Se integra en Q y K de cada capa de atencion, manteniendo el resto de componentes del Transformer.	Contexto de esta prueba El maximo efectivo de contexto sigue siendo 128 tokens; no se evalua aqui extrapolacion a longitudes mayores.

Resultado observado

RoPE (azul) mejora de manera clara y sostenida el validation loss frente a V4.5 Looped Transformer (verde). La separacion aparece pronto y aumenta durante el entrenamiento; al final de la grafica el loss es aproximadamente 5.8 con RoPE frente a aproximadamente 6.4 con la referencia, una diferencia visual cercana a 0.6 puntos.

Interpretacion y decision

La mejora es una de las mas marcadas de las iteraciones registradas. Mantener RoPE en la configuracion principal y recuperar del log los valores exactos de loss final y mejor checkpoint. La atribucion al cambio es mas fiable si se confirma que todos los demas hiperparametros y la semilla se mantuvieron fijos frente a V4.5.

Siguiente experimento recomendado

Separar dos preguntas: (1) comparacion RoPE frente a positional embeddings a contexto fijo de 128; (2) capacidad de generalizacion al aumentar la longitud de contexto. La segunda requiere evaluar longitudes mayores, no solo reutilizar esta curva de entrenamiento.

Evidencia - V4.7

Validation loss: RoPE frente a V4.5 Looped Transformer. Azul: RoPE; verde: Looped Transformer.



Lectura de la figura

RoPE baja mas rapidamente tras la fase inicial y mantiene una ventaja creciente. La diferencia final visual es grande para las escalas observadas y justifica priorizar esta configuracion en el desarrollo posterior.

Precaucion de reporte

Los valores 5.8 y 6.4 son estimaciones visuales de la grafica. Para el informe tecnico, sustituirlos por las metricas exportadas del logger y acompañarlos de la configuracion y seed exactas.

Entrada de bitacora 08

Implementacion V4.8 - vocabulario reducido a 16,384 IDs

Fecha: no registrada Autor/a: _____ Estado: mejora observada con caveat de comparabilidad

Problema detectado

La configuracion anterior usaba el vocabulario de GPT-2 de 50,257 IDs. El coste de parametros no pertenece al tokenizador como objeto, sino principalmente a las matrices del modelo dependientes del vocabulario: embedding de entrada y, segun se comparta o no peso, proyeccion de salida/softmax. Con emb_dim = 480, una sola matriz de embedding de GPT-2 ya supone aproximadamente 24.1 M de parametros.

Cambio y presupuesto de parametros

Aspecto	GPT-2	V4.8: tokenizer pequeno
Tamano del vocabulario	50,257 IDs	16,384 IDs
Matriz de embedding (480 dim)	Aprox. 24.1 M parametros	Aprox. 7.9 M parametros
Parametros totales del modelo	Aprox. 35 M	Aprox. 18 M
Capacidad no ligada al vocabulario	Aprox. 9-10 M para MLP y atencion, segun el desglose registrado	Mas presupuesto relativo para los bloques internos o un modelo total mas pequeno

Resultado observado

Pese a reducir el numero total de parametros, V4.8 con vocabulario de 16,384 IDs (morado) mejora claramente el validation/test loss respecto a V4.7 con el vocabulario GPT-2 (azul). La curva se separa pronto y termina alrededor de 4.8 frente a alrededor de 6.1 en la grafica, una mejora visual muy grande dentro de esta medicion.

Interpretacion y limitacion esencial

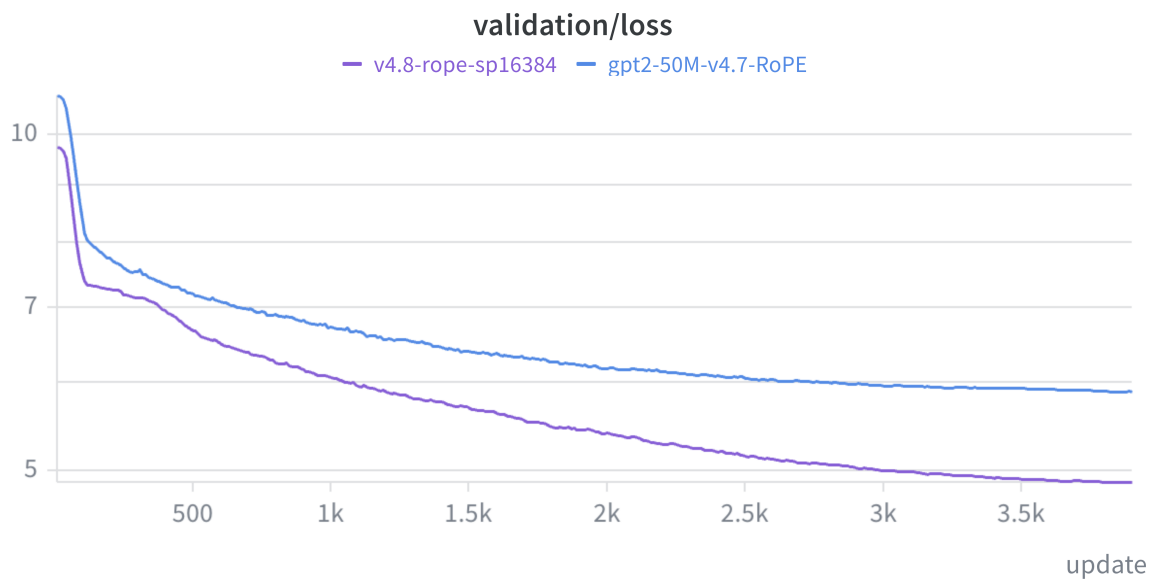
Comparabilidad de la metrica. Al cambiar el tokenizador, una misma secuencia de texto se divide en un numero y unos tipos de tokens diferentes. Por eso, el cross-entropy/loss por token no es estrictamente comparable entre vocabularios. La mejora observada es prometedora, pero el informe debe complementarla con una metrica normalizada sobre el texto original, por ejemplo bits por byte (BPB), y con evaluaciones de generacion o tareas comunes.

Decision y siguiente paso

Mantener el vocabulario pequeno como candidato principal por eficiencia de parametros y mejora observada. Verificar que la tokenizacion se entreno y aplico sin fuga de datos, calcular BPB en el mismo texto de validacion y registrar tokens por caracter, coste y throughput junto con el loss.

Evidencia - V4.8

Validation loss: vocabulario de 16,384 IDs frente al vocabulario GPT-2. Morado: V4.8; azul: V4.7 con RoPE.



Lectura de la figura

La curva con vocabulario reducido cae antes y continua por debajo de V4.7 durante toda la corrida. Es la mayor separacion observada en las graficas hasta este punto, aun cuando el modelo total tiene menos parametros.

Como reportarlo correctamente

Reportar esta figura como evidencia de mejor loss bajo cada tokenizacion, no como una comparacion definitiva de calidad por texto. Añadir BPB o una evaluacion comun antes de concluir que el vocabulario reducido es mejor en sentido absoluto.

Entrada de bitacora 09

Fecha: _____ Autor/a: _____ Duracion: _____

Objetivo de esta sesion

Que querias averiguar, construir o corregir?

Cambios realizados

Codigo, datos, configuracion, dependencias o infraestructura.

Evidencia y resultado

Incluye metricas, tabla o enlace a grafica. Separa el hecho de la interpretacion.

Decision y siguiente paso

Que mantienes, cambias o descartas? Que haras a continuacion?

Registro de experimento

Usa una hoja por ejecucion o por conjunto de ejecuciones que respondan a la misma pregunta.

ID del experimento _____	Fecha / commit _____
Pregunta / hipotesis	Baseline con el que comparo
Datos Version, particion, preprocesado.	Modelo Arquitectura, parametros, inicializacion.
Configuracion LR, batch size, tokens, seed, scheduler.	Recursos GPU/CPU, tiempo, memoria, coste.
Resultado cuantitativo	Resultado cualitativo / muestras
Interpretacion y confianza	Conclusion / siguiente accion

Registro de decisiones

Las decisiones pequeñas también cuentan: cambiar un tokenizer, descartar una métrica o congelar una versión de datos.

Fecha	Decision	Alternativas y evidencia	Impacto / seguimiento

Incidencias y aprendizajes

Problema:

Causa probable:

Como se resolvió / que se aprendió:

Como prevenirlo:

Del cuaderno al informe tecnico

Al cerrar el proyecto, no redactes desde la memoria. Agrupa las entradas por estas secciones y enlaza cada afirmacion con evidencia.

Seccion del informe	Material del cuaderno que alimenta esa seccion	Comprobacion final
Resumen ejecutivo	Objetivo, resultado principal y conclusion de las entradas.	Una frase con impacto y una metrica.
Problema y alcance	Ficha del proyecto, riesgos y exclusiones.	Evita prometer lo que no se evaluo.
Metodologia	Registros de experimento, datos, modelo y configuracion.	Permite reproducir el resultado.
Resultados	Metricas, graficas, muestras y comparacion con baseline.	Incluye resultados negativos relevantes.
Discusion	Interpretaciones, incidencias y limites anotados.	Distingue evidencia de opinion.
Conclusiones y futuro	Decisiones finales y siguientes pasos pendientes.	Lista acciones concretas y priorizadas.

Lista de cierre

Antes de escribir: congela la version de codigo y datos, exporta las graficas finales, conserva las semillas/configuraciones y selecciona los 3-5 experimentos que mejor responden a la pregunta original.

Codigo relevante - Bitacoras 01 y 02

Fragmentos seleccionados del codigo de implementacion. Se incluyen solo las lineas que materializan cada cambio; los notebooks y modulos indicados conservan el contexto completo.

Bitacora 01 - configuracion y baseline

Fuente: notebooks/Test_pretrain-v4-wandb.ipynb

```
MAX_LENGTH = 128
BATCH_SIZE = 2
MAX_TOKENS = 1_000_000
MAX_UPDATES = MAX_TOKENS // (BATCH_SIZE * MAX_LENGTH)
LEARNING_RATE = 3e-4

cfg = {'**GPT_CONFIG_50M', 'context_length': MAX_LENGTH}
model = GPTModel(cfg)
model.out_head.weight = model.tok_emb.weight
```

Bitacora 02 - warm-up y cosine decay

Fuente: src/llm_mini_lab/training/core.py

```
def cosine_lr_multiplier(step, warmup_steps, schedule_steps,
                        min_lr_ratio=0.1):
    if step < warmup_steps:
        return (step + 1) / warmup_steps
    progress = (step - warmup_steps) / (schedule_steps - warmup_steps)
    cosine = 0.5 * (1 + math.cos(math.pi * min(progress, 1.0)))
    return min_lr_ratio + (1 - min_lr_ratio) * cosine

WARMUP_STEPS = int(0.05 * MAX_UPDATES)
scheduler = torch.optim.lr_scheduler.LambdaLR(
    optimizer, lambda step: cosine_lr_multiplier(
        step, WARMUP_STEPS, MAX_UPDATES, 0.1))
```

Codigo relevante - Bitacoras 03 y 04

Bitacora 03 - inicializacion Xavier

Fuente: src/llm_mini_lab/training/core.py

```
def init_xavier(module):
    if isinstance(module, torch.nn.Linear):
        torch.nn.init.xavier_uniform_(module.weight)
        if module.bias is not None:
            torch.nn.init.zeros_(module.bias)
    elif isinstance(module, torch.nn.Embedding):
        torch.nn.init.xavier_uniform_(module.weight)

model = GPTModel(cfg)
model.apply(init_xavier)
model.out_head.weight = model.tok_emb.weight
```

Bitacora 04 - MLP con SwiGLU

Fuente: src/llm_mini_lab/models/layers.py

```
class SwiGLU(nn.Module):
    def __init__(self, emb_dim, hidden_dim):
        self.gate_proj = nn.Linear(emb_dim, hidden_dim)
        self.value_proj = nn.Linear(emb_dim, hidden_dim)
        self.out_proj = nn.Linear(hidden_dim, emb_dim)

    def forward(self, x):
        gate = torch.nn.functional.silu(self.gate_proj(x))
        value = self.value_proj(x)
        return self.out_proj(gate * value)

hidden_dim = cfg.get("ff_hidden_dim", int(8 * emb_dim / 3))
self.layers = SwiGLU(emb_dim=emb_dim, hidden_dim=hidden_dim)
```

Codigo relevante - Bitacoras 05 y 06

Bitacora 05 - variante de mayor profundidad

Fuente: notebooks/Test_pretrain-v4.4_more_depht.ipynb

```
GPT_CONFIG_50M = {
    "vocab_size": 50257, "context_length": 256, "emb_dim": 480,
    "n_heads": 4,        # antes: 8
    "n_layers": 9,       # antes: 7
    "drop_rate": 0.0, "qkv_bias": False,
    "ff_activation": "swiglu", "ff_hidden_dim": 1280,
}
cfg = {**GPT_CONFIG_50M, "context_length": MAX_LENGTH,
      "ff_activation": "swiglu", "ff_hidden_dim": 1376}
```

Bitacora 06 - bloques Transformer recurrentes

Fuente: src/llm_mini_lab/models/gpt.py

```
self.trf_blocks = nn.ModuleList([
    TransformerBlock(cfg) for _ in range(cfg["n_unique_layers"])
])
self.num_loops = cfg["num_loops"]

loops = self.num_loops if num_loops is None else num_loops
for _ in range(loops):
    for block in self.trf_blocks:
        x = block(x)
```

Codigo de bitacoras 07 y 08

Bitacora 07 - Rotary Positional Embeddings

Fuente: src/llm_mini_lab/models/layers.py

```
def _apply_rope(self, x):
    cos = self.rope_cos[:x.size(-2)].unsqueeze(0).unsqueeze(0)
    sin = self.rope_sin[:x.size(-2)].unsqueeze(0).unsqueeze(0)
    even, odd = x[..., 0::2], x[..., 1::2]
    return torch.stack((even * cos - odd * sin,
                        even * sin + odd * cos), dim=-1).flatten(-2)

if self.use_rope:
    keys = self._apply_rope(keys)
    queries = self._apply_rope(queries)
```

Bitacora 08 - tokenizer de 16,384 IDs y RoPE

Fuente: notebooks/v4.8_low_size_token.ipynb

```
TOKENIZER_NAME = "sp16384"
TOKENIZER_MODEL = PROJECT_ROOT / "tokenizers" / "fineweb_16384_bpe.model"
tokenizer = load_tokenizer(TOKENIZER_NAME, TOKENIZER_MODEL)
VOCAB_SIZE = tokenizer_vocab_size(tokenizer)

cfg = {**LOOPED_GPT_CONFIG, "context_length": MAX_LENGTH,
      "vocab_size": VOCAB_SIZE, "positional_encoding": "rope",
      "tokenizer_name": TOKENIZER_NAME}
model = LoopedGPTModel(cfg).to(device)
model.out_head.weight = model.tok_emb.weight
```